Software Task Guide

A simple, reusable template: use it for any feature, bug fix, or change.

1. Task Brief (fill in first)

Task title	
Task type	Feature / Bug fix / Refactor / Docs / Other
Owner	
Deadline	
Goal (1-2 lines)	What should be true when this is done?
Out of scope	What will NOT be done in this task?

2. Ground Rules

1	**One task, one purpose.** Do not mix unrelated changes in the same piece of work.
2	**Understand before you build.** If the requirement is unclear, ask before writing code.
3	**Work in small steps.** Commit often with clear messages (what changed and why).
4	**Never commit secrets.** API keys, passwords, and tokens stay in environment variables.
5	**Test your own work.** Nothing is done until you have run it and seen it work.
6	**Keep it readable.** Clear names, short functions, comments only where the reason is not obvious.
7	**Do not break existing behavior.** Run existing tests before and after your change.
8	**Ask for review.** A second pair of eyes catches what you cannot see.

3. Step-by-Step Workflow

#	Step	What to do	Done when
1	**Understand**	Read the task. Write the goal in one sentence. List inputs, outputs, and constraints.	Goal is clear and written down
2	**Plan**	Break the work into small subtasks. Note files, tools, and risks. Estimate time.	Subtask list exists
3	**Set up**	Pull latest code, create a branch (e.g. feature/short-name), confirm the project runs locally.	Clean branch, app runs
4	**Build**	Implement one subtask at a time. Commit after each working piece.	Each subtask works on its own
5	**Test**	Run existing tests, add new ones, and try edge cases (empty input, wrong input, large input).	All tests pass
6	**Review**	Re-read your diff. Remove debug code. Check naming, errors, and docs. Request peer review.	Diff is clean, feedback addressed

#	Step	What to do	Done when
7	**Deliver**	Merge or deploy per team process. Update docs and notify stakeholders.	Change is live and communicated
8	**Reflect**	Note what went well, what slowed you down, and what to do differently next time.	Short retro note saved

4. Definition of Done (checklist)

[ ]	Requirement understood and goal written
[ ]	Branch created from latest code
[ ]	Code runs locally without errors
[ ]	New tests added; all tests pass
[ ]	Edge cases and error handling checked
[ ]	No secrets, debug logs, or dead code left
[ ]	Documentation / README updated if needed
[ ]	Peer review completed
[ ]	Merged / deployed and stakeholders informed

5. Progress Log

Date	Update / blocker	Next action

6. When You Get Stuck

- Reproduce the problem with the smallest possible example.
- Read the error message fully; the answer is often in it.
- Change one thing at a time, then re-test.
- Timebox it: after 30 minutes with no progress, ask for help and share what you tried.